LangChain & LangGraph

Weeks 7 & 8 -- Day-by-Day Study Plan

Week 7: Production & API		Week 8: Testing, Eval & Portfolio	

Day 43	FastAPI Setup -- wrap LangGraph agent in async FastAPI app
Day 44	Streaming Endpoint -- SSE /stream endpoint, real-time tokens
Day 45	Frontend UI -- minimal HTML+JS chat interface calling your API
Day 46	Load Testing & Auth -- httpx concurrency test, API key middleware
Day 47	pytest for LangGraph -- unit test individual nodes, mock LLM calls
Day 48	LangSmith Evaluation -- 30+ Q&A dataset, automated eval pipeline
Day 49	RAGAs + Open Source -- RAG evaluation metrics, annotate GitHub project
Day 50	Portfolio Project -- design, build & document your own end-to-end agent

PREREQUISITES

Before starting Week 7, confirm you have completed:

Weeks 1-4: LLM fundamentals, LangChain, LangGraph basics, checkpointing

Week 5-6: Multi-agent systems, parallelism, error handling, real projects

pip install fastapi uvicorn httpx pytest ragas

A working LangGraph agent from Week 6 to wrap in an API

LangSmith account + API key already set up from Week 3

Week 7 -- Production & API

Day 43 . FastAPI Setup -- Wrapping LangGraph in an API

(1) THEORY -- TOPICS TO COVER

FastAPI basics: routes, request/response models, async handlers
Pydantic models for validating incoming chat requests
LangGraph async execution: ainvoke for non-blocking API responses
CORS middleware -- allowing browser frontends to call your API
Uvicorn ASGI server -- running FastAPI locally and reloading on changes
Managing LangGraph state across HTTP requests with thread IDs

(2) FOCUS / SKIP GUIDE

FOCUS ON

The /chat POST endpoint: receives {message, thread_id} -> returns {response}
Using ainvoke so FastAPI doesn't block while LangGraph runs
Thread IDs from the request body: each user session gets its own thread
Pydantic ChatRequest model -- FastAPI validates input automatically
Testing with curl and httpx before building any frontend

DON'T WORRY ABOUT YET

Production deployment (Docker, Kubernetes, cloud) -- local uvicorn is enough
Database-backed user sessions and authentication -- Day 46
Rate limiting and quotas -- focus on getting it working first
OpenAPI spec generation -- FastAPI does this automatically, explore after

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] FastAPI official documentation
-> https://fastapi.tiangolo.com/
[Docs] FastAPI -- First steps tutorial
-> https://fastapi.tiangolo.com/tutorial/first-steps/
[YouTube] Karan Bhanot -- FastAPI + LangChain streaming
-> https://www.youtube.com/watch?v=x3nkGhANK2Y
[Docs] LangGraph deployment options guide
-> https://langchain-ai.github.io/langgraph/concepts/deployment_options/
[Docs] FastAPI -- Async/await patterns
-> https://fastapi.tiangolo.com/async/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] pip install fastapi uvicorn httpx python-multipart
- [ ] Create main.py with a FastAPI app and a GET /health endpoint
- [ ] Define a Pydantic ChatRequest model: message: str, thread_id: str
- [ ] Add POST /chat endpoint that calls your Week 6 agent with ainvoke
- [ ] Add CORS middleware to allow requests from localhost:3000
- [ ] Run with: uvicorn main:app --reload -- confirm it starts

- [ ] Test /chat with curl and with httpx in a Python script

TASK RESOURCES

[Docs] FastAPI -- Request body with Pydantic
-> https://fastapi.tiangolo.com/tutorial/body/
[Docs] FastAPI -- CORS middleware
-> https://fastapi.tiangolo.com/tutorial/cors/
[Docs] FastAPI -- async route handlers
-> https://fastapi.tiangolo.com/async/
[Docs] Uvicorn -- running and reloading
-> https://www.uvicorn.org/

(4) CODE EXAMPLES

main.py -- FastAPI + LangGraph /chat endpoint

```
from fastapi import FastAPI
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.sqlite.aio import AsyncSqliteSaver

app = FastAPI(title='LangGraph API', version='1.0')

app.add_middleware(
    CORSMiddleware,
    allow_origins=['*'],
    allow_methods=['*'],
    allow_headers=['*'],
)

class ChatRequest(BaseModel):
    message:   str
    thread_id: str = 'default'

class ChatResponse(BaseModel):
    response:  str
    thread_id: str

@app.get('/health')
async def health(): return {'status': 'ok'}

@app.post('/chat', response_model=ChatResponse)
async def chat(req: ChatRequest):
    async with AsyncSqliteSaver.from_conn_string('agent.db') as ckpt:
        agent = build_agent(ckpt)  # your compiled LangGraph graph
        cfg   = {'configurable': {'thread_id': req.thread_id}}
        result = await agent.ainvoke(
            {'messages': [HumanMessage(req.message)]},
            config=cfg
        )
        return ChatResponse(
            response=result['messages'][-1].content,
            thread_id=req.thread_id
        )
```

Test with httpx

```
# test_api.py -- run with: python test_api.py
import httpx, asyncio

async def test_chat():
    async with httpx.AsyncClient(base_url='http://localhost:8000') as client:
        # Health check
        r = await client.get('/health')
        print('Health:', r.json())

        # Single turn
        r = await client.post('/chat', json={
            'message': 'What is RAG?',
            'thread_id': 'test_session_1'
        })
        print('Response:', r.json()['response'][:100])

        # Follow-up -- same thread_id, agent remembers context
        r = await client.post('/chat', json={
            'message': 'Can you give me a code example?',
            'thread_id': 'test_session_1'
        })
        print('Follow-up:', r.json()['response'][:100])

asyncio.run(test_chat())
```

READ MORE -- TOPIC REFERENCES

[Docs] FastAPI -- automatic API docs at /docs
-> https://fastapi.tiangolo.com/tutorial/first-steps/#interactive-api-docs
[Docs] LangGraph -- LangGraph Server concepts
-> https://langchain-ai.github.io/langgraph/concepts/langgraph_server/
[Docs] FastAPI -- background tasks
-> https://fastapi.tiangolo.com/tutorial/background-tasks/

(5) DAY COMPLETION CHECKLIST

- [ ] FastAPI app starts with uvicorn main:app --reload
- [ ] GET /health returns {'status': 'ok'}
- [ ] POST /chat accepts ChatRequest and returns ChatResponse
- [ ] ainvoke used -- endpoint does not block while agent runs
- [ ] Thread IDs work -- two requests with same thread_id share memory
- [ ] CORS middleware enabled -- no CORS errors from browser
- [ ] Tested with both curl and httpx -- both work correctly
- [ ] Can explain: why ainvoke is needed instead of invoke in FastAPI

Day 44 . Streaming Endpoint -- SSE for Real-Time Tokens

(1) THEORY -- TOPICS TO COVER

Server-Sent Events (SSE) -- one-way server push over HTTP
FastAPI StreamingResponse with EventSourceResponse

LangGraph astream_events -- emitting token-by-token from graph nodes
on_chat_model_stream events -- extracting token chunks from event stream
Client-side EventSource API -- connecting to SSE from JavaScript
Why SSE over WebSockets for LLM streaming (simpler, HTTP-native)

(2) FOCUS / SKIP GUIDE

FOCUS ON

The generator pattern: async def event_generator() yield 'data: ...\n\n'
astream_events version='v2' -- the stable API for token streaming
Filtering events: only on_chat_model_stream contains actual tokens
StreamingResponse with media_type='text/event-stream'
Testing SSE with curl -N so you see tokens arrive one by one

DON'T WORRY ABOUT YET

WebSocket bidirectional streaming -- SSE is simpler and enough for LLMs
Reconnection logic and event IDs for SSE fault tolerance
Backpressure handling for very slow clients
Compressing SSE streams for bandwidth saving

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] FastAPI -- StreamingResponse docs
-> https://fastapi.tiangolo.com/advanced/custom-response/#streamingresponse
[MDN] MDN -- Server-Sent Events (EventSource API)
-> https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events
[YouTube] Patrick Loeber -- FastAPI streaming with SSE
-> https://www.youtube.com/watch?v=OElJWtHOEtE
[Docs] LangGraph -- astream_events reference
-> https://langchain-ai.github.io/langgraph/how-tos/astream_events/
[PyPI] sse-starlette package for SSE in FastAPI
-> https://pypi.org/project/sse-starlette/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] pip install sse-starlette
- [ ] Add GET /stream endpoint that takes message and thread_id as query params
- [ ] Write an async generator that calls astream_events and yields tokens
- [ ] Wrap generator in EventSourceResponse from sse-starlette
- [ ] Test with curl -N 'http://localhost:8000/stream?message=Hello&thread;_id=t1'
- [ ] Confirm tokens arrive one-by-one in the terminal
- [ ] Add a [DONE] sentinel event after the stream completes

TASK RESOURCES

[Docs] LangGraph -- streaming tokens from LLM nodes
-> https://langchain-ai.github.io/langgraph/how-tos/streaming-tokens/
[Docs] sse-starlette -- EventSourceResponse usage
-> https://github.com/sysid/sse-starlette
[Docs] FastAPI -- async generators for streaming
-> https://fastapi.tiangolo.com/advanced/custom-response/#streamingresponse

(4) CODE EXAMPLES

/stream SSE endpoint

```
from fastapi.responses import StreamingResponse
from sse_starlette.sse import EventSourceResponse
import json

@app.get('/stream')
async def stream_chat(message: str, thread_id: str = 'default'):
    async def event_generator():
        async with AsyncSqliteSaver.from_conn_string('agent.db') as ckpt:
            agent = build_agent(ckpt)
            cfg   = {'configurable': {'thread_id': thread_id}}
            async for event in agent.astream_events(
                {'messages': [HumanMessage(message)]},
                config=cfg,
                version='v2'
            ):
                if event['event'] == 'on_chat_model_stream':
                    token = event['data']['chunk'].content
                    if token:
                        yield {'data': json.dumps({'token': token})}
            yield {'data': json.dumps({'token': '[DONE]'})}
    return EventSourceResponse(event_generator())
```

Test SSE with curl and httpx

```
# Terminal test -- watch tokens arrive one by one
# curl -N 'http://localhost:8000/stream?message=Explain+RAG&thread_id=t1'

# Python test with httpx streaming
import httpx, asyncio, json

async def test_stream():
    url = 'http://localhost:8000/stream'
    params = {'message': 'Explain RAG in simple terms', 'thread_id': 'stream_test'}
    async with httpx.AsyncClient(timeout=60) as client:
        async with client.stream('GET', url, params=params) as resp:
            async for line in resp.aiter_lines():
                if line.startswith('data:'):
                    data = json.loads(line[5:].strip())
                    token = data.get('token', '')
                    if token == '[DONE]':
                        print()  # newline at end
                        break
                    print(token, end='', flush=True)

asyncio.run(test_stream())
```

READ MORE -- TOPIC REFERENCES

[Docs] LangGraph -- astream_events v2 event types reference
-> https://langchain-ai.github.io/langgraph/how-tos/astream_events/
[MDN] MDN -- Using server-sent events
-> https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

(5) DAY COMPLETION CHECKLIST

- [ ] GET /stream endpoint added -- accepts message and thread_id query params

- [ ] async generator yields {'data': json} for each token
- [ ] EventSourceResponse wraps the generator correctly
- [ ] curl -N test shows tokens arriving one by one in terminal
- [ ] [DONE] sentinel emitted after stream completes
- [ ] httpx async streaming test reads and prints the full streamed response
- [ ] Can explain: what SSE is and why it is preferred over WebSockets for LLMs

---

Day 45 . Frontend UI -- Minimal HTML+JS Chat Interface

(1) THEORY -- TOPICS TO COVER

EventSource JavaScript API -- connecting to SSE from the browser
fetch() for non-streaming POST /chat calls
DOM manipulation: appending message bubbles dynamically
Serving static HTML from FastAPI with StaticFiles mount
UX basics for chat: scroll-to-bottom, disable input while streaming

(2) FOCUS / SKIP GUIDE

FOCUS ON

The EventSource pattern: new EventSource(url) -> onmessage handler
Appending tokens to a message bubble as they arrive (streaming effect)
Sending thread_id from the browser to maintain conversation state
Serving the HTML file via FastAPI so there is no CORS issue
Testing the full stack: browser -> FastAPI -> LangGraph -> SSE -> browser

DON'T WORRY ABOUT YET

React, Vue, or any frontend framework -- plain HTML+JS is enough
CSS polish, animations, or mobile responsiveness
User accounts, login pages, or persistent chat history UI
WebSocket connections -- SSE is sufficient for one-way streaming

RESOURCES -- WATCH / READ BEFORE STARTING

[MDN] MDN -- EventSource (SSE) JavaScript API
-> https://developer.mozilla.org/en-US/docs/Web/API/EventSource
[Docs] FastAPI -- StaticFiles for serving HTML
-> https://fastapi.tiangolo.com/tutorial/static-files/
[YouTube] Traversy Media -- Fetch API crash course
-> https://www.youtube.com/watch?v=Oive66jrwBs
[MDN] MDN -- fetch() API reference
-> https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Create static/index.html with a text input, send button, and chat div
- [ ] Mount the static folder in FastAPI with app.mount('/static', StaticFiles(...))
- [ ] Add JavaScript: on send, call GET /stream with EventSource
- [ ] Append each token to the current bot message bubble as it arrives

- [ ] When [DONE] received, enable input and scroll to bottom
- [ ] Add a thread_id stored in localStorage so conversation persists on refresh
- [ ] Test full round-trip: type message -> see streaming response in browser

TASK RESOURCES

[Docs] FastAPI -- StaticFiles mount
-> https://fastapi.tiangolo.com/tutorial/static-files/
[MDN] MDN -- localStorage API
-> https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage
[MDN] MDN -- scrollIntoView for auto-scroll
-> https://developer.mozilla.org/en-US/docs/Web/API/Element/scrollIntoView

(4) CODE EXAMPLES

Mount static files in FastAPI

```
from fastapi.staticfiles import StaticFiles
import os

# Create static/ folder if it doesn't exist
os.makedirs('static', exist_ok=True)

# Mount static files -- serves index.html at /static/index.html
app.mount('/static', StaticFiles(directory='static'), name='static')

# Optional: redirect / to the chat UI
from fastapi.responses import RedirectResponse
@app.get('/')
async def root(): return RedirectResponse('/static/index.html')
```

static/index.html -- streaming chat UI

```
<!-- static/index.html -->
<!DOCTYPE html>
<html><head><title>LangGraph Chat</title></head>
<body>
  <div id='chat' style='height:400px;overflow-y:auto;border:1px solid #ccc;padding:10px'></div>
  <input id='input' type='text' placeholder='Ask something...' style='width:80%'>
  <button id='send'>Send</button>
  <script>
    const chat = document.getElementById('chat');
    const input = document.getElementById('input');
    const send = document.getElementById('send');
    let threadId = localStorage.getItem('threadId') || crypto.randomUUID();
    localStorage.setItem('threadId', threadId);
    function addMessage(role, text) {
      const div = document.createElement('div');
      div.style.margin = '8px 0';
      div.innerHTML = '<b>' + role + ':</b> ' + text;
      chat.appendChild(div);
      chat.scrollTop = chat.scrollHeight;
      return div;
    }
    send.onclick = () => {
      const msg = input.value.trim();
      if (!msg) return;
      addMessage('You', msg);
```

READ MORE -- TOPIC REFERENCES

[Docs] FastAPI -- Jinja2 templates (for more dynamic HTML)
-> https://fastapi.tiangolo.com/advanced/templates/
[Blog] LangChain -- Building a streaming chat UI
-> https://blog.langchain.dev/streaming-chat-ui/

(5) DAY COMPLETION CHECKLIST

- [ ] static/index.html created with input, send button, and chat display
- [ ] StaticFiles mounted in FastAPI -- file accessible at /static/index.html
- [ ] EventSource connects to /stream -- tokens appear one by one in the browser
- [ ] thread_id stored in localStorage -- conversation continues on page refresh
- [ ] Input disabled during streaming, re-enabled on [DONE]
- [ ] Auto-scroll keeps the latest message visible
- [ ] Full round-trip tested: type -> stream -> see response in browser

---

Day 46 . Load Testing & Basic Auth -- Production Hardening

(1) THEORY -- TOPICS TO COVER

Concurrent HTTP load testing with httpx.AsyncClient
API key authentication using FastAPI HTTPBearer dependency
Request rate limiting concepts -- protecting your LLM budget
Async connection pooling -- why it matters for concurrent requests
Health check and readiness patterns for production services
Environment variables for secrets -- never hardcode API keys

(2) FOCUS / SKIP GUIDE

FOCUS ON

Running 10 concurrent /chat requests with asyncio.gather -- measuring response time
HTTPBearer dependency: extract Bearer token from Authorization header
Validating the token against an env var SECRET_API_KEY
Returning 401 Unauthorized when token is missing or wrong
Using python-dotenv to load API keys from .env in FastAPI

DON'T WORRY ABOUT YET

OAuth2, JWT tokens, or database-backed user accounts -- too complex for now
Nginx rate limiting or cloud API gateway throttling
Distributed load balancing across multiple FastAPI instances
Full k6 or Locust load test suites -- simple httpx test is fine

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] FastAPI -- Security: HTTP Basic / Bearer
-> https://fastapi.tiangolo.com/tutorial/security/
[Docs] FastAPI -- Dependencies for auth
-> https://fastapi.tiangolo.com/tutorial/dependencies/
[Docs] httpx -- Async client concurrency
-> https://www.python-httpx.org/async/
[YouTube] ArjanCodes -- FastAPI authentication patterns
-> https://www.youtube.com/watch?v=0_seNFCtglk

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Write a load test: send 10 concurrent /chat requests with asyncio.gather
- [ ] Record min/max/avg response times -- print a summary table
- [ ] Add HTTPBearer dependency to /chat and /stream endpoints
- [ ] Validate the Bearer token against SECRET_API_KEY from environment
- [ ] Return HTTP 401 if token is missing or incorrect
- [ ] Test: correct token -> 200, wrong token -> 401, no token -> 401
- [ ] Add a /ready endpoint that checks the database file exists

TASK RESOURCES

[Docs] FastAPI -- HTTPBearer security scheme
-> https://fastapi.tiangolo.com/tutorial/security/http-basic-auth/
[Docs] FastAPI -- Dependencies in path operations
-> https://fastapi.tiangolo.com/tutorial/dependencies/
[Docs] httpx -- async gather pattern
-> https://www.python-httpx.org/async/

(4) CODE EXAMPLES

Load test -- 10 concurrent requests

```
import httpx, asyncio, time

API_KEY = 'test-secret-key'
HEADERS = {'Authorization': f'Bearer {API_KEY}'}

async def single_request(client, i):
    start = time.time()
    r = await client.post('/chat',
        json={'message': f'Hello {i}, what is RAG?', 'thread_id': f'load_{i}'},
        headers=HEADERS
    )
    elapsed = time.time() - start
    return elapsed, r.status_code

async def load_test(concurrency=10):
    async with httpx.AsyncClient(base_url='http://localhost:8000', timeout=60) as client:
        tasks   = [single_request(client, i) for i in range(concurrency)]
        results = await asyncio.gather(*tasks)
    times   = [r[0] for r in results]
    statuses= [r[1] for r in results]
    print(f'Requests: {concurrency}')
    print(f'All 200: {all(s==200 for s in statuses)}')
```

API key authentication dependency

```
from fastapi import Depends, HTTPException, status
from fastapi.security import HTTPBearer, HTTPAuthorizationCredentials
import os

security = HTTPBearer()
SECRET_API_KEY = os.getenv('SECRET_API_KEY', 'dev-secret-key')

def verify_token(credentials: HTTPAuthorizationCredentials = Depends(security)):
    if credentials.credentials != SECRET_API_KEY:
        raise HTTPException(
            status_code=status.HTTP_401_UNAUTHORIZED,
            detail='Invalid API key',
            headers={'WWW-Authenticate': 'Bearer'},
        )
    return credentials.credentials

# Protect /chat with auth dependency
@app.post('/chat', response_model=ChatResponse)
async def chat(req: ChatRequest, token: str = Depends(verify_token)):
    # ... same as before ...
    pass

# Readiness check
@app.get('/ready')
async def ready():
    import pathlib
    if pathlib.Path('agent.db').exists():
        return {'status': 'ready'}
    raise HTTPException(503, 'Database not initialised')
```

READ MORE -- TOPIC REFERENCES

[Docs] FastAPI -- OAuth2 with Password (next step after Bearer)
-> https://fastapi.tiangolo.com/tutorial/security/oauth2-jwt/
[Blog] Real Python -- FastAPI best practices for production
-> https://realpython.com/fastapi-python-web-apis/

(5) DAY COMPLETION CHECKLIST

- [ ] Load test runs 10 concurrent requests -- all return 200
- [ ] Min/max/avg response times printed -- no timeouts
- [ ] HTTPBearer dependency added to /chat and /stream
- [ ] Correct token -> 200, wrong token -> 401, no token -> 401
- [ ] SECRET_API_KEY loaded from environment variable
- [ ] GET /ready returns 200 when DB exists, 503 otherwise
- [ ] Week 7 API is complete: /health, /chat, /stream, /ready all working

Week 8 -- Testing, Evaluation & Portfolio

Day 47 . pytest for LangGraph -- Unit Testing Agent Nodes

(1) THEORY -- TOPICS TO COVER

pytest basics: test functions, assert statements, fixtures
Unit testing individual LangGraph nodes in isolation
Mocking LLM calls with unittest.mock to avoid API costs in tests
Testing state transitions: assert the correct fields change
Testing conditional routing: assert the right node name is returned
pytest-asyncio for testing async node functions

(2) FOCUS / SKIP GUIDE

FOCUS ON

Testing nodes independently -- pass a state dict, assert the output dict
Mocking llm.invoke() with MagicMock returning a fake AIMessage
Testing routing functions: call with different state -> assert correct route
Testing the compiled graph with invoke -- integration-level test
Keeping tests fast: mock all LLM and external API calls

DON'T WORRY ABOUT YET

End-to-end browser testing with Selenium or Playwright
Test coverage metrics and CI/CD pipelines -- focus on writing tests first
Property-based testing with Hypothesis
Performance benchmarking tests -- measure latency separately

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] pytest official docs
-> https://docs.pytest.org/
[Docs] pytest -- fixtures tutorial
-> https://docs.pytest.org/en/stable/how-to/fixtures.html
[Docs] pytest-asyncio -- testing async code
-> https://pytest-asyncio.readthedocs.io/
[YouTube] ArjanCodes -- pytest complete guide
-> https://www.youtube.com/watch?v=cHYq1MRoyI0
[Docs] Python -- unittest.mock reference
-> https://docs.python.org/3/library/unittest.mock.html

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] pip install pytest pytest-asyncio
- [ ] Write unit tests for your Week 6 classify_intent node
- [ ] Mock llm.invoke() with MagicMock returning a fake AIMessage
- [ ] Write unit tests for all routing functions -- test every branch
- [ ] Write an integration test for the full compiled graph with ainvoke
- [ ] Run pytest -v and fix any failing tests

- [ ] Aim for all nodes and all routing branches covered

TASK RESOURCES

[Docs] pytest -- how to monkeypatch
-> https://docs.pytest.org/en/stable/how-to/monkeypatch.html
[Docs] pytest-asyncio -- async test examples
-> https://pytest-asyncio.readthedocs.io/en/latest/
[Docs] LangChain -- FakeLLM for testing
-> https://python.langchain.com/docs/integrations/llms/fake/

(4) CODE EXAMPLES

Unit testing LangGraph nodes with mocked LLM

```
# tests/test_nodes.py
import pytest
from unittest.mock import MagicMock, patch
from langchain_core.messages import AIMessage, HumanMessage

# Test classify_intent node (no LLM needed -- pure Python)
def test_classify_order_intent():
    state = {'messages': [HumanMessage('Where is my order ORD001?')], 'intent': ''}
    result = classify_intent(state)  # your node function
    assert result['intent'] == 'order'

def test_classify_faq_intent():
    state = {'messages': [HumanMessage('What is your return policy?')], 'intent': ''}
    result = classify_intent(state)
    assert result['intent'] == 'faq'

# Test routing function -- all branches
def test_route_intent_all_branches():
    assert route_intent({'intent': 'order'})    == 'order_agent'
    assert route_intent({'intent': 'faq'})      == 'faq_agent'
    assert route_intent({'intent': 'complaint'})== 'complaint_agent'
    assert route_intent({'intent': 'general'})  == 'general_agent'

# Test node that calls LLM -- mock the LLM
@patch('your_module.llm')
def test_agent_node_with_mock_llm(mock_llm):
    fake_response = AIMessage(content='This is a mocked LLM response')
    mock_llm.invoke.return_value = fake_response
    state = {'messages': [HumanMessage('Test query')], 'intent': 'general'}
    result = general_agent_node(state)
    assert len(result['messages']) > 0
    mock_llm.invoke.assert_called_once()
```

Integration test for compiled graph

```
import pytest, asyncio
from langchain_core.messages import HumanMessage
from langgraph.checkpoint.memory import MemorySaver

@pytest.mark.asyncio
async def test_full_graph_integration():
    # Use MemorySaver to avoid touching real DB in tests
    memory = MemorySaver()
    app    = build_agent(checkpointer=memory)  # your compiled graph
```

READ MORE -- TOPIC REFERENCES

[Docs] LangChain -- FakeChatModel for testing without API costs
-> https://python.langchain.com/docs/integrations/chat/fake/
[Blog] LangChain -- Testing LangChain applications
-> https://blog.langchain.dev/testing-langchain-applications/

(5) DAY COMPLETION CHECKLIST

- [ ] pytest installed -- pytest -v runs without errors
- [ ] Unit tests for classify_intent -- all intent branches covered
- [ ] Unit tests for all routing functions -- every branch tested
- [ ] LLM mocked with MagicMock -- tests run without any API calls
- [ ] Integration test for compiled graph with MemorySaver passes
- [ ] All tests pass with pytest -v
- [ ] Can explain: unit test vs integration test and when to use each

---

Day 48 . LangSmith Evaluation -- Automated Eval Pipeline

(1) THEORY -- TOPICS TO COVER

LangSmith evaluation dataset: inputs, reference outputs, metadata
Evaluator types: LLM-as-judge, string distance, custom Python evaluators
Tracking eval results across versions: comparing scores over time
Structured rubrics for LLM-as-judge: criteria-based scoring 0-1
Automating evals in CI: re-run eval on every code change
Reading the eval results pandas DataFrame and interpreting scores

(2) FOCUS / SKIP GUIDE

FOCUS ON

Building a dataset of 30+ Q&A; pairs that covers all agent capabilities
Writing a rubric: 'Does the answer correctly address the question? 1=yes, 0=no'
Running evaluate() and reading the results table in LangSmith UI
Comparing two runs: version A vs version B -- did your changes improve scores?
Identifying the lowest-scoring examples -- those are your weakest cases

DON'T WORRY ABOUT YET

Automated CI/CD eval triggers via GitHub Actions -- manual runs are fine for now
Statistical significance testing across eval runs
Custom embedding-based semantic similarity evaluators
Multi-dimensional rubrics with 5+ criteria -- keep it to 2-3 criteria

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] LangSmith evaluation how-to (official)
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application
[Docs] LangSmith -- Creating and managing datasets
-> https://docs.smith.langchain.com/how_to_guides/datasets
[YouTube] LangChain -- LangSmith evaluation tutorial
-> https://www.youtube.com/watch?v=Hab2CV_0hpQ
[Docs] LangSmith -- LLM-as-judge evaluators reference
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application#use-llm-as-judge
[Blog] LangChain -- Evaluating RAG pipelines
-> https://blog.smith.langchain.com/evaluating-rag-pipelines-with-langsmith

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Create a LangSmith dataset with 30+ Q&A; pairs covering all agent use cases
- [ ] Write 3 LLM-as-judge evaluators: correctness, relevance, completeness
- [ ] Run evaluate() on your Week 6 agent -- record baseline scores
- [ ] Make a small improvement to one agent node or prompt
- [ ] Re-run evaluate() -- compare new scores to baseline in LangSmith UI
- [ ] Find the 5 lowest-scoring examples -- diagnose what went wrong
- [ ] Export results as pandas DataFrame and print a summary

TASK RESOURCES

[Docs] LangSmith -- evaluate() function reference
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application
[Docs] LangSmith -- Custom evaluator functions
-> https://docs.smith.langchain.com/how_to_guides/evaluation/evaluate_llm_application#custom-evaluators
[Docs] LangSmith -- Comparing experiment results
-> https://docs.smith.langchain.com/how_to_guides/evaluation/compare_experiments

(4) CODE EXAMPLES

30-item dataset + 3 LLM-as-judge evaluators

```
from langsmith import Client
from langsmith.evaluation import evaluate, LangChainStringEvaluator
from langchain_core.prompts import ChatPromptTemplate

client = Client()

# Build dataset from your test cases
dataset = client.create_dataset('agent-eval-v1', description='Week 6 agent eval')
examples = [
    {'inputs': {'message': 'Where is my order ORD001?', 'thread_id': 'e1'},
     'outputs': {'expected': 'Shipped - arrives Thursday'}},
    {'inputs': {'message': 'What is your return policy?', 'thread_id': 'e2'},
     'outputs': {'expected': 'Our return policy allows returns within 30 days'}},
    # ... add 28 more pairs ...
]
client.create_examples(dataset_id=dataset.id, examples=examples)

# Define the pipeline under test
def run_agent(inputs):
```

READ MORE -- TOPIC REFERENCES

[Docs] LangSmith -- Annotation queues for human labelling
-> https://docs.smith.langchain.com/how_to_guides/human_feedback
[Blog] LangChain -- Building eval pipelines that scale
-> https://blog.smith.langchain.com/llm-eval-best-practices

(5) DAY COMPLETION CHECKLIST

- [ ] LangSmith dataset with 30+ Q&A; pairs created
- [ ] 3 evaluators defined: correctness, relevance, and one custom
- [ ] Baseline eval run completed -- scores recorded
- [ ] One improvement made to agent -- re-evaluated and scores compared
- [ ] 5 lowest-scoring examples identified and diagnosed
- [ ] Results exported as pandas DataFrame -- mean scores printed
- [ ] Can explain: what LLM-as-judge is and when it is better than exact match

---

Day 49 . RAGAs + Open Source -- RAG Metrics & Reading Real Projects

(1) THEORY -- TOPICS TO COVER

RAGAs library -- automated evaluation metrics specific to RAG pipelines
RAGAs metrics: faithfulness, answer relevancy, context recall, context precision
Why RAG evaluation is hard: retrieval quality AND generation quality both matter
Reading open-source LangGraph projects: understanding real production patterns
Contributing to open source: filing issues, reading code, understanding PRs

(2) FOCUS / SKIP GUIDE

FOCUS ON

Faithfulness: does the answer only use information from the retrieved context?
Answer relevancy: does the answer actually address what was asked?
Running RAGAs on your Week 2/3 RAG pipeline with a test set of 10+ questions
Reading one real LangGraph project on GitHub end-to-end (README + main graph file)
Taking notes: what patterns does it use that you have not seen before?

DON'T WORRY ABOUT YET

Contributing code to LangGraph itself -- reading and understanding is enough
All RAGAs metrics at once -- pick faithfulness + answer_relevancy to start
Fine-tuning embedding models based on RAGAs context recall scores
RAGAs with non-English datasets

RESOURCES -- WATCH / READ BEFORE STARTING

[Docs] RAGAs documentation (official)
-> https://docs.ragas.io/
[Docs] RAGAs -- getting started tutorial
-> https://docs.ragas.io/en/stable/getstarted/rag_testset_generation/
[YouTube] LangChain -- RAGAs evaluation tutorial
-> https://www.youtube.com/watch?v=h0DHDp1FbmQ
[GitHub] LangGraph examples on GitHub
-> https://github.com/langchain-ai/langgraph/tree/main/examples
[Docs] LangChain -- Evaluating RAG with RAGAs
-> https://python.langchain.com/docs/how_to/evaluate_rag/

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] pip install ragas
- [ ] Build a RAGAs evaluation dataset: 10+ questions with retrieved context + answers
- [ ] Run faithfulness and answer_relevancy metrics on your RAG pipeline
- [ ] Find the 3 lowest-scoring questions -- diagnose why they failed
- [ ] Browse LangGraph examples on GitHub -- pick one project to read closely
- [ ] Read the chosen project end-to-end: state, nodes, edges, tools
- [ ] Write 5 bullet notes: what patterns did you learn from this project?

TASK RESOURCES

[Docs] RAGAs -- metrics reference
-> https://docs.ragas.io/en/stable/concepts/metrics/index.html
[Docs] RAGAs -- evaluate() function
-> https://docs.ragas.io/en/stable/getstarted/evaluation/
[GitHub] LangGraph open-source examples directory
-> https://github.com/langchain-ai/langgraph/tree/main/examples

(4) CODE EXAMPLES

RAGAs evaluation on your RAG pipeline

```
from ragas import evaluate
from ragas.metrics import faithfulness, answer_relevancy, context_recall
from datasets import Dataset

# Build evaluation dataset
eval_data = {
    'question': [
        'What is retrieval-augmented generation?',
        'How does chunking affect RAG quality?',
        'What is the difference between dense and sparse retrieval?',
    ],
    'answer': [],      # fill with your RAG pipeline outputs
    'contexts': [],    # fill with retrieved chunks for each question
    'ground_truth': [  # reference answers
        'RAG combines retrieval of relevant documents with LLM generation.',
        'Smaller chunks improve precision; larger chunks improve context.',
        'Dense uses embeddings; sparse uses keyword matching like BM25.',
    ]
}
```

READ MORE -- TOPIC REFERENCES

[Docs] RAGAs -- context precision and recall explained
-> https://docs.ragas.io/en/stable/concepts/metrics/index.html
[GitHub] LangGraph real-world examples (official repo)
-> https://github.com/langchain-ai/langgraph/tree/main/examples
[Blog] LangChain -- RAG evaluation best practices
-> https://blog.smith.langchain.com/evaluating-rag-pipelines-with-langsmith

(5) DAY COMPLETION CHECKLIST

- [ ] RAGAs installed -- from ragas import evaluate works
- [ ] Evaluation dataset built with 10+ questions, answers, contexts, ground_truth
- [ ] faithfulness and answer_relevancy scores computed and printed
- [ ] 3 lowest-scoring questions identified and failure reason written down
- [ ] One LangGraph open-source project read end-to-end on GitHub
- [ ] 5 bullet notes written: new patterns learned from the real project
- [ ] Can explain: faithfulness vs answer_relevancy -- what each metric measures

---

Day 50 . Portfolio Project -- Your Own End-to-End Agent

(1) THEORY -- TOPICS TO COVER

Portfolio project design: problem statement, agent architecture, tools needed
Writing a clear README: what it does, how to run it, what you learned
Git workflow: commits, README, .env.example, requirements.txt
Agent architecture decision: LangChain chain vs LangGraph graph vs multi-agent
Presenting your work: demo video or GIF, architecture diagram, key learnings

(2) FOCUS / SKIP GUIDE

FOCUS ON

Choosing a problem you genuinely care about -- motivation matters
Start with a one-page design doc: state, nodes, tools, API shape
Building the MVP first: minimum viable agent that works end-to-end
Writing the README as if a stranger needs to run it in 5 minutes
The architecture diagram: export from LangGraph draw_mermaid()

DON'T WORRY ABOUT YET

Building something novel or research-level -- practical usefulness is enough
Perfect code quality on first pass -- ship a working MVP, refactor later
Deploying to the cloud -- local uvicorn + GitHub repo is a complete portfolio piece
Making it production-ready -- document known limitations honestly

RESOURCES -- WATCH / READ BEFORE STARTING

[Blog] LangChain -- LangGraph use case gallery
-> https://www.langchain.com/use-cases
[GitHub] LangGraph examples for inspiration
-> https://github.com/langchain-ai/langgraph/tree/main/examples
[Blog] LangChain -- Building your first production agent
-> https://blog.langchain.dev/langgraph-for-production/
[Docs] LangGraph Platform -- overview of deployment options
-> https://langchain-ai.github.io/langgraph/concepts/deployment_options/
[YouTube] LangChain -- Building real-world LLM applications
-> https://www.youtube.com/watch?v=R0XGQHL-4LM

(3) PRACTICAL -- TASKS TO COMPLETE

- [ ] Write a 1-page design doc: problem, state design, nodes, tools, API shape
- [ ] Set up a new Git repo with .env.example and requirements.txt
- [ ] Build the MVP: working end-to-end agent with at least 3 nodes
- [ ] Wrap in FastAPI with /chat and /stream endpoints
- [ ] Add SqliteSaver for persistence and at least one interrupt checkpoint
- [ ] Write 5+ pytest unit tests covering nodes and routing
- [ ] Add LangSmith tracing -- run a full eval with 10+ test cases
- [ ] Export Mermaid diagram and include in README
- [ ] Write a clear README: what, why, how to run, architecture, known limits
- [ ] Record a 2-min demo video or create a GIF showing the agent in action

TASK RESOURCES

[Tool] Mermaid Live -- paste your draw_mermaid() output
-> https://mermaid.live
[Docs] GitHub -- Writing a good README guide
-> https://docs.github.com/en/repositories/managing-your-repositorys-settings-and-features/customizing-your-repository/about-readmes
[Tool] LiceCap -- free GIF screen recorder (Mac/Windows)
-> https://www.cockos.com/licecap/
[Docs] LangGraph -- deployment options for sharing
-> https://langchain-ai.github.io/langgraph/concepts/deployment_options/

(4) CODE EXAMPLES

Portfolio project structure template

```
# Recommended project layout
my_agent/
  main.py          # FastAPI app
  agent.py         # LangGraph graph definition
  tools.py         # @tool decorated functions
  prompts.py       # system prompts and templates
  tests/
    test_nodes.py  # unit tests for each node
    test_api.py    # integration tests for /chat /stream
  static/
    index.html     # minimal chat frontend
  .env.example    # OPENAI_API_KEY=... SECRET_API_KEY=...
```

README.md template

```
# My LangGraph Agent -- [Project Name]

## What it does
[1-2 sentences: describe the problem and what your agent solves]

## Architecture
![Graph diagram](docs/graph.png)
[Brief description of each node and why you designed it this way]

## How to run
git clone https://github.com/you/my-agent
cd my-agent
cp .env.example .env  # add your OPENAI_API_KEY
pip install -r requirements.txt
uvicorn main:app --reload
# Open http://localhost:8000 in your browser

## Running tests
pytest tests/ -v --asyncio-mode=auto

## What I learned
[3-5 honest bullet points about challenges and insights]

## Known limitations
[Be honest -- what does it not handle well yet?]
```

READ MORE -- TOPIC REFERENCES

[GitHub] Awesome LangChain -- curated project examples
-> https://github.com/kyrolabs/awesome-langchain
[Blog] LangChain -- LangGraph production tips
-> https://blog.langchain.dev/langgraph-for-production/
[Docs] LangGraph Platform -- managed deployment option
-> https://langchain-ai.github.io/langgraph/concepts/langgraph_platform/

(5) DAY COMPLETION CHECKLIST

- [ ] Design doc written: problem, state, nodes, tools, API shape all defined
- [ ] Git repo set up with .env.example and requirements.txt
- [ ] MVP built: agent runs end-to-end with at least 3 nodes
- [ ] FastAPI /chat and /stream endpoints working
- [ ] SqliteSaver persistence + at least one interrupt_before checkpoint
- [ ] 5+ pytest tests passing with mocked LLM calls
- [ ] LangSmith tracing on + eval dataset with 10+ test cases run
- [ ] Mermaid diagram exported and included in README
- [ ] README written: clear setup instructions + architecture + learnings
- [ ] Demo video or GIF recorded showing the agent working
- [ ] COURSE COMPLETE -- 8 weeks done, portfolio project shipped!